Project submission PART 8

Covid-19 cases – Team B

Government Arts College salem-7

Name: Soundhar.M

Mail id: soundhar991999@gmail.com

Module 1:

COVID-19 CASES

PROBLEM DEFINITION:

- Large number of data little bit difficult for handling.
- The pandemic has already taken grip over peoples' life. Since the start of the pandemic, some countries are facing problem of ever-increasing cases
- Through the data analysis of cases one can analyse how countries all over the world are doing in terms of controlling the pandemic.
- Predictions are made with the dataset available to the individual/country/organisations, thus helping them to decide how far they are able to control the pandemic or up to how much extent they should guide preventive measures.
- Through this project, a step towards helping people to understand the spread and predict the cases in their country is done.

DESIGN THINKING:

- A global pandemic puts enormous stress on governments and healthcare services. Suddenly, there is a scramble to circulate the correct information and roll out products and services to deal with the crisis.
- **These challenges bring together a blend of product design, experience design, and service design problems that are desperate for a solution, and design thinking can help.**
- The design thinking process consists of five stages: **empathize, define, ideate, prototype, and test.**

- To use Data Wrangling process the raw data converting to usable form. It is easily to understandable and ready-to-use formats, with methods varying from project to project.

Module 2:

PROBLEM INNOVATION AND PROBLEM SOLVING

1.create a Github

2.

```
Try:
   File_Obj =("Covid_19_cases4.txt","r");
   File_Content = File _Object.read();
File_content.close;
#context manager
With open("Covid_19_cases4.txt","r");file_content = File_Obj.read();
Except Exception as ex:
Print(ex);
```

3.Function,classes

4.Comment section

5.Try to create separate python file for different usage (File segregation)

6.(SOLID)

S-Single Responsibility principles

O-open/close principles

L-lisko substitution principes

I-Interface segregation principles

D-Dependency inversion principles

Module 3:

1.Project Title: Covid19-cases

Problem Statement: You are tasked with creating a Python project to analyze COVID-19 cases data. The project should be able to read data from a CSV file, process and analyze the data, and generate reports and visualizations. The data file should contain information about

COVID-19 cases such as the date, country, number of confirmed cases, number of deaths, and number of recoveries. The project should be able to handle data from multiple countries.

Project Steps:

1.Data Collection: Collect data on patients with covid-19 from various sources such as hospitals, medical journals, and public datasets.

Dataset Link:

https://www.kaggle.com/datasets/chakradharmattapalli/covid-19-cases

2.Data-pre-processing: Clean the collected data, remove any irrelevant or missing values and normalize the data.

3.data Visualization: Visualing Data for deep understanding.

4.Model Selection: Train various machine learning models on the selected features to predict the covid 19 cases.

5.Model Evaluation: Evaluate the performance of the models using various metrics such as accuracy, precision and recall.

Conclusion: The proposed project aims to develop a machine learning based model that can Analyse the the covid19 pandemic effect.

6.

```
import pandas as pd

# Load the COVID-19 data into a pandas DataFrame
covid_data = pd.read_csv('path/to/covid_data.csv')

# Drop unnecessary columns
covid_data = covid_data.drop(columns=['Lat', 'Long'])

# Rename columns for consistency
covid_data = covid_data.rename(columns={
    'Province/State': 'state',
    'Country/Region': 'country',
```

```
    'Date': 'date',
    'Confirmed': 'confirmed',
    'Deaths': 'deaths',
    'Recovered': 'recovered'
})

# Group by country and date, and sum the confirmed, deaths, and recovered cases
covid_data = covid_data.groupby(['country', 'date']).agg({
    'confirmed': 'sum',
    'deaths': 'sum',
    'recovered': 'sum'
}).reset_index()

# Convert date column to datetime format
covid_data['date'] = pd.to_datetime(covid_data['date'])

# Add a column for active cases
covid_data['active'] = covid_data['confirmed'] - covid_data['deaths'] - covid_data['recovered']

# Save the cleaned data to a new file
covid_data.to_csv('path/to/cleaned_covid_data.csv', index=False)
```

Module 4:

Module 4: Data Analysis

Step1: Read the dataset

Step2: Data cleaning

Step 3: Data analysis in various ways

Code:

```
import pandas as pd
import numpy as np
import seaborn as sns

data = pd.read_csv("D:/Covid_19_cases.csv")

#understanding the data

data.head()
```

	dateRep	day	month	year	cases	deaths	countriesAndTerritories
0	31-05-2021	31	5	2021	366	5	Austria
1	30-05-2021	30	5	2021	570	6	Austria
2	29-05-2021	29	5	2021	538	11	Austria
3	28-05-2021	28	5	2021	639	4	Austria
4	27-05-2021	27	5	2021	405	19	Austria

```
data.tail()
```

	dateRep	day	month	year	cases	deaths	countriesAndTerritories
2725	06-03-2021	6	3	2021	3455	17	Sweden
2726	05-03-2021	5	3	2021	4069	12	Sweden
2727	04-03-2021	4	3	2021	4884	14	Sweden
2728	03-03-2021	3	3	2021	4876	19	Sweden
2729	02-03-2021	2	3	2021	6191	19	Sweden

```
data.shape
```

(2730, 7)

data.describe()

	day	month	year	cases	deaths
count	2730.000000	2730.000000	2730.0	2730.000000	2730.000000
mean	16.000000	4.010989	2021.0	3661.010989	65.291941
std	8.765919	0.818813	0.0	6490.510073	113.956634
min	1.000000	3.000000	2021.0	-2001.000000	-3.000000
25%	8.000000	3.000000	2021.0	361.250000	2.000000
50%	16.000000	4.000000	2021.0	926.500000	14.500000
75%	24.000000	5.000000	2021.0	3916.250000	72.000000
max	31.000000	5.000000	2021.0	53843.000000	956.000000

data.columns

```
Index(['dateRep', 'day', 'month', 'year', 'cases', 'deaths',
       'countriesAndTerritories'],
      dtype='object')
```

data.nunique()

```
dateRep                  91
day                      31
month                     3
year                      1
cases                  1902
deaths                  381
countriesAndTerritories     30
dtype: int64
data['day'].unique()
array([31, 30, 29, 28, 27, 26, 25, 24, 23, 22, 21, 20, 19, 18, 17, 16, 15,
       14, 13, 12, 11, 10,  9,  8,  7,  6,  5,  4,  3,  2,  1],
      dtype=int64)
```

data['countriesAndTerritories'].unique()

```
array(['Austria', 'Belgium', 'Bulgaria', 'Croatia', 'Cyprus', 'Czechia',
       'Denmark', 'Estonia', 'Finland', 'France', 'Germany', 'Greece',
       'Hungary', 'Iceland', 'Ireland', 'Italy', 'Latvia',
       'Liechtenstein', 'Lithuania', 'Luxembourg', 'Malta', 'Netherlands',
       'Norway', 'Poland', 'Portugal', 'Romania', 'Slovakia', 'Slovenia',
       'Spain', 'Sweden'], dtype=object)
```

#cleaning the data

```
data.isnull().sum()
```

```
dateRep                  0
day                      0
month                    0
year                     0
cases                    0
deaths                   0
countriesAndTerritories  0
dtype: int64
```

```
Covid_19_cases = data.drop(['dateRep', 'countriesAndTerritories'],axis=1)
```

```
Covid_19_cases.head()
```

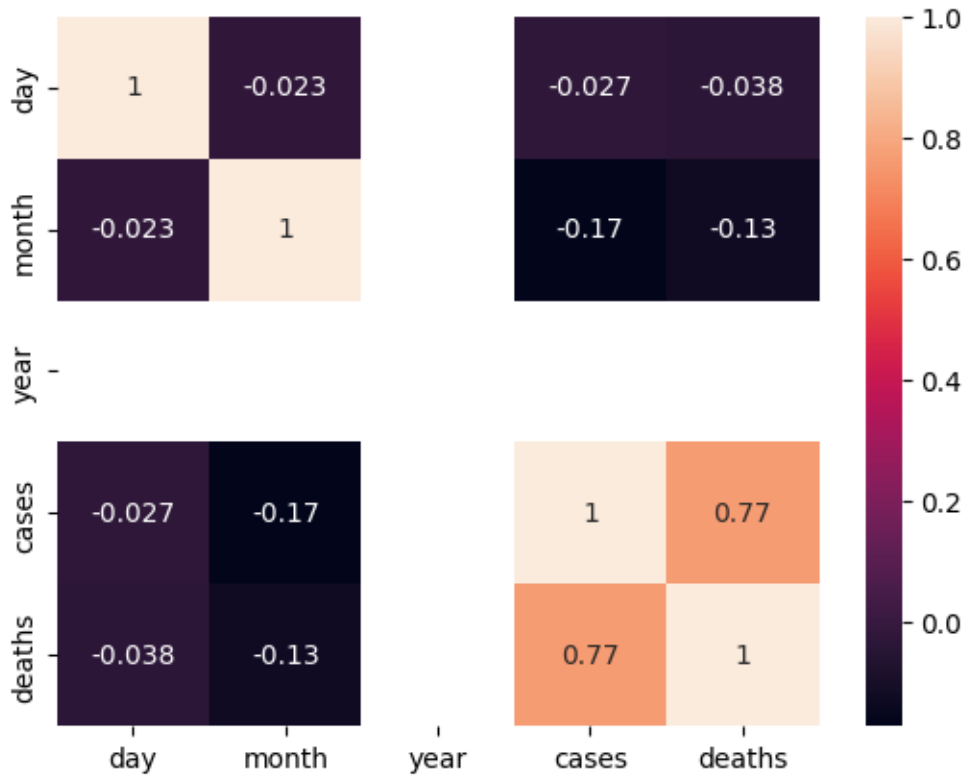
	day	month	year	cases	deaths
0	31	5	2021	366	5
1	30	5	2021	570	6
2	29	5	2021	538	11
3	28	5	2021	639	4
4	27	5	2021	405	19

 # relationship analysis

```
corelation = Covid_19_cases.corr()
```

```
sns.heatmap(corelation, xticklabels=corelation.columns, yticklabels=corelation.columns, annot=True)
```

```
<Axes: >
```

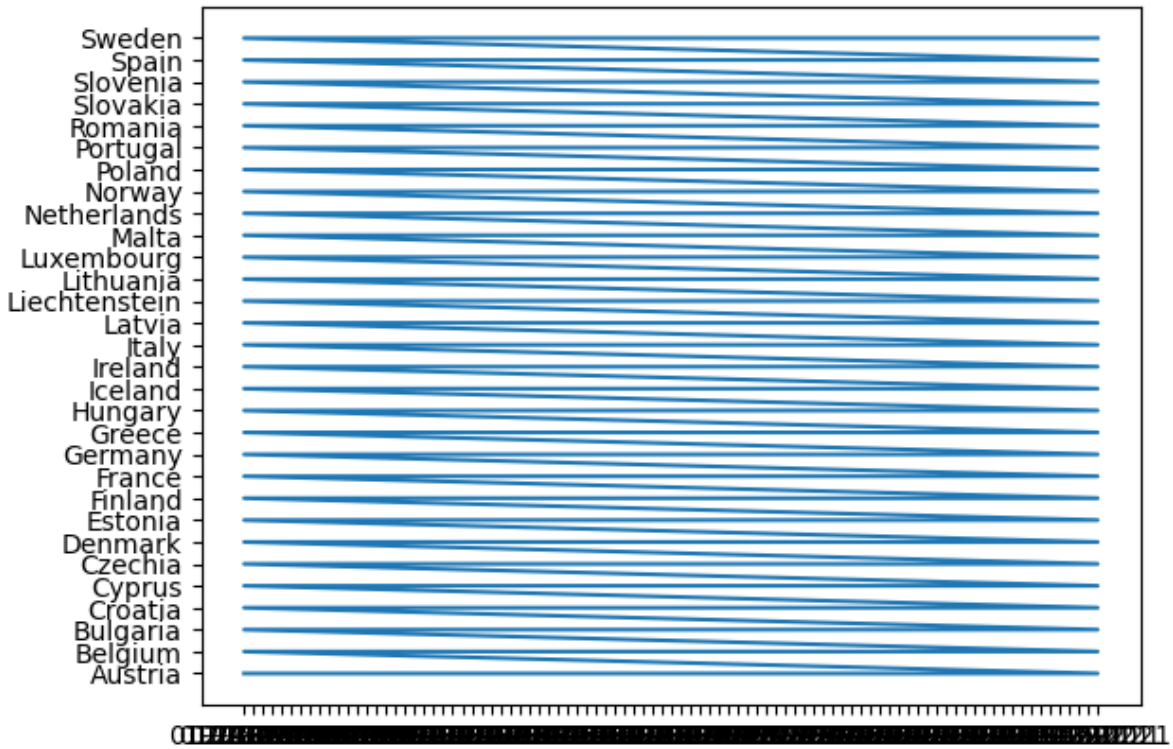



```
sns.pairplot(Covid_19_cases)
```

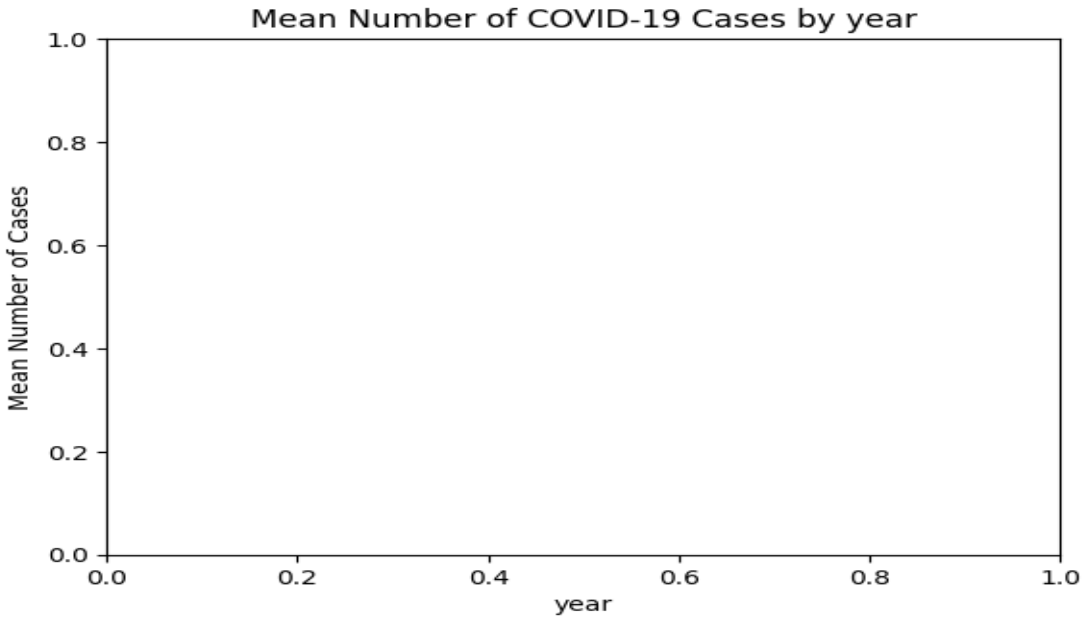
<seaborn.axisgrid.PairGrid at 0x1b470e40110>

Module 5:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Read in the data and convert the 'day' column to a datetime format
df = pd.read_csv('D:/Covid_19_cases4.csv')
df['day'] = pd.to_datetime(df['day'], format='%d')

# Group the data by day and calculate the mean for each column
df_mean = df.groupby(['countriesAndTerritories', 'dateRep'], as_index=False).mean()

# Convert date column to datetime format
df['year'] = pd.to_datetime(df['year'], format='%Y')

print(df.columns)
Index(['dateRep', 'day', 'month', 'year', 'cases', 'deaths',
       'countriesAndTerritories'],
      dtype='object')

# Create a line plot of the mean number of cases by day
plt.plot(df_mean['dateRep'], df_mean['countriesAndTerritories'])

[<matplotlib.lines.Line2D at 0x1ac7e21b210>]
```

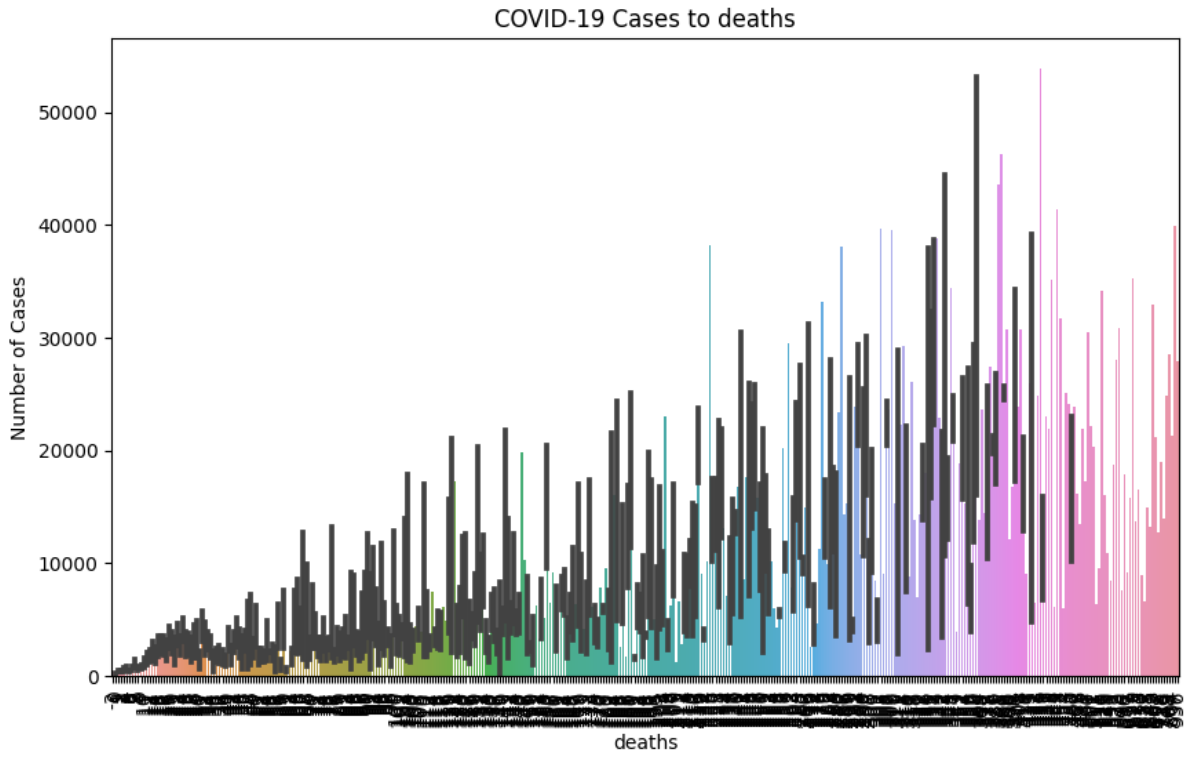



```
# Add labels and title
plt.xlabel('year')
plt.ylabel('Mean Number of Cases')
plt.title('Mean Number of COVID-19 Cases by year')
# Display the plot
plt.show()
```

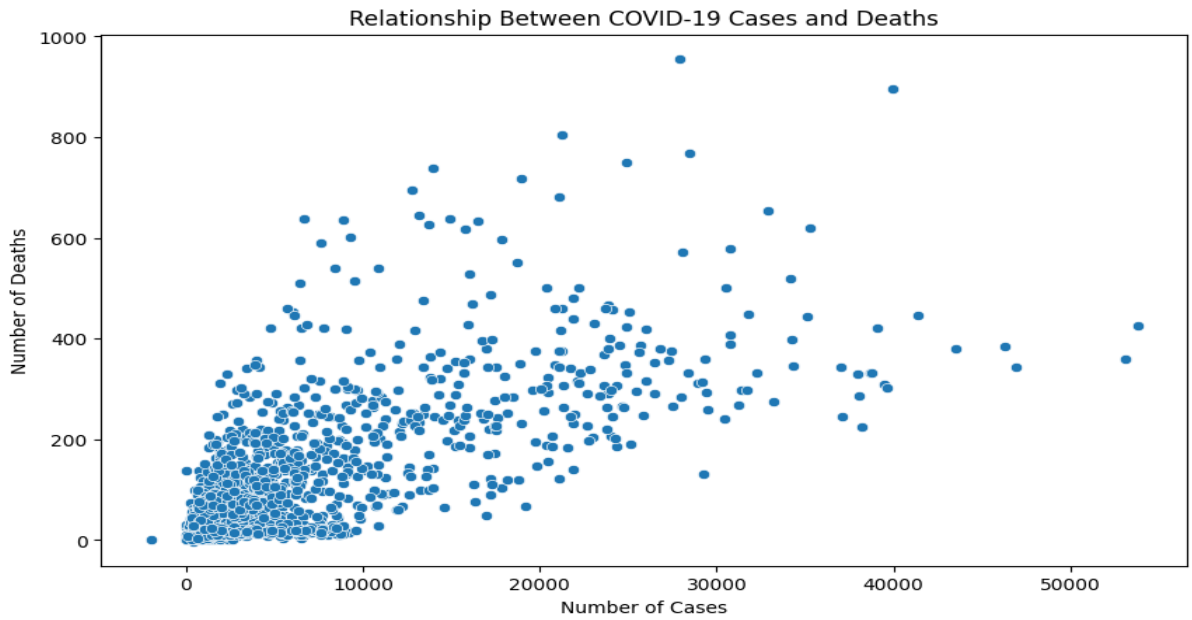



```
# Create a bar chart to show the number of cases by country
plt.figure(figsize=(10,6))
sns.barplot(x='deaths', y='cases', data=df)
plt.xlabel('deaths')
plt.ylabel('Number of Cases')
```

```
plt.title('COVID-19 Cases to deaths')
plt.xticks(rotation=90)
plt.show()
```




```
# Create a scatterplot to show the relationship between cases and deaths
plt.figure(figsize=(10,6))
sns.scatterplot(x='cases', y='deaths', data=df)
plt.xlabel('Number of Cases')
plt.ylabel('Number of Deaths')
plt.title('Relationship Between COVID-19 Cases and Deaths')
plt.show()
```



:

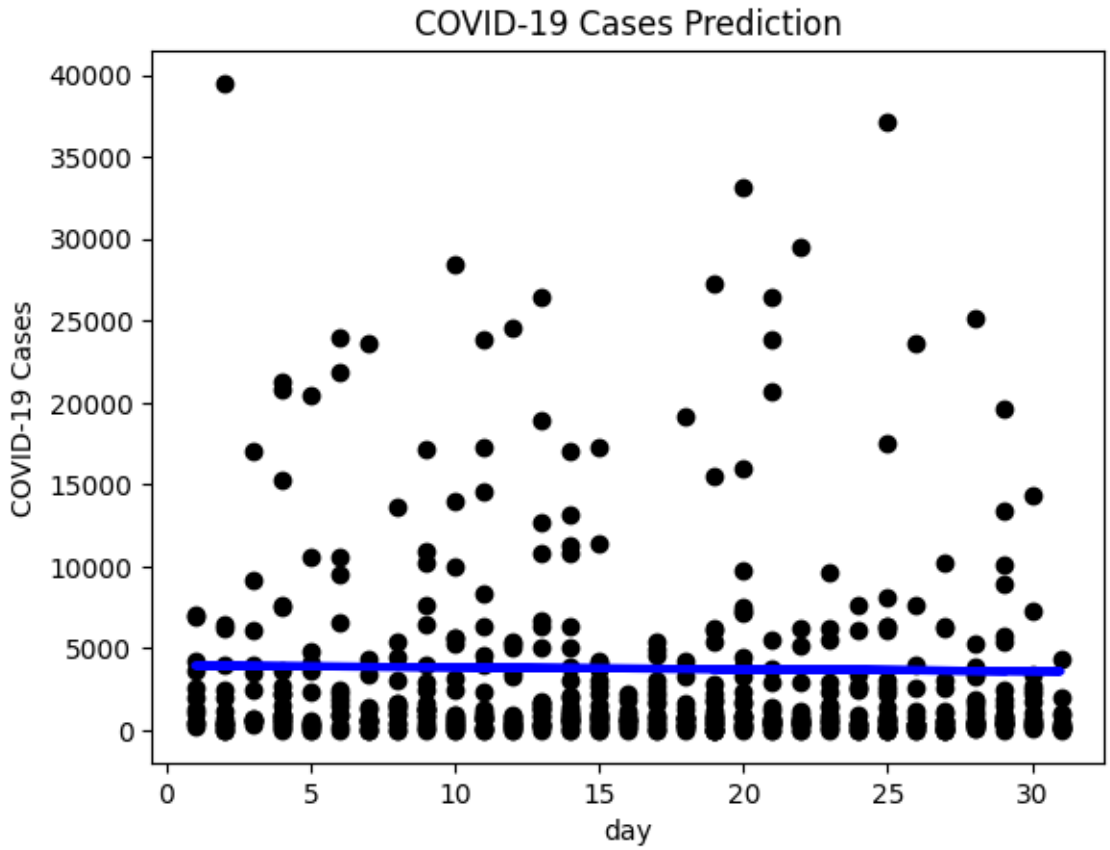
```
y_data= x_data * 0.1+0.3
print(y_data)
```

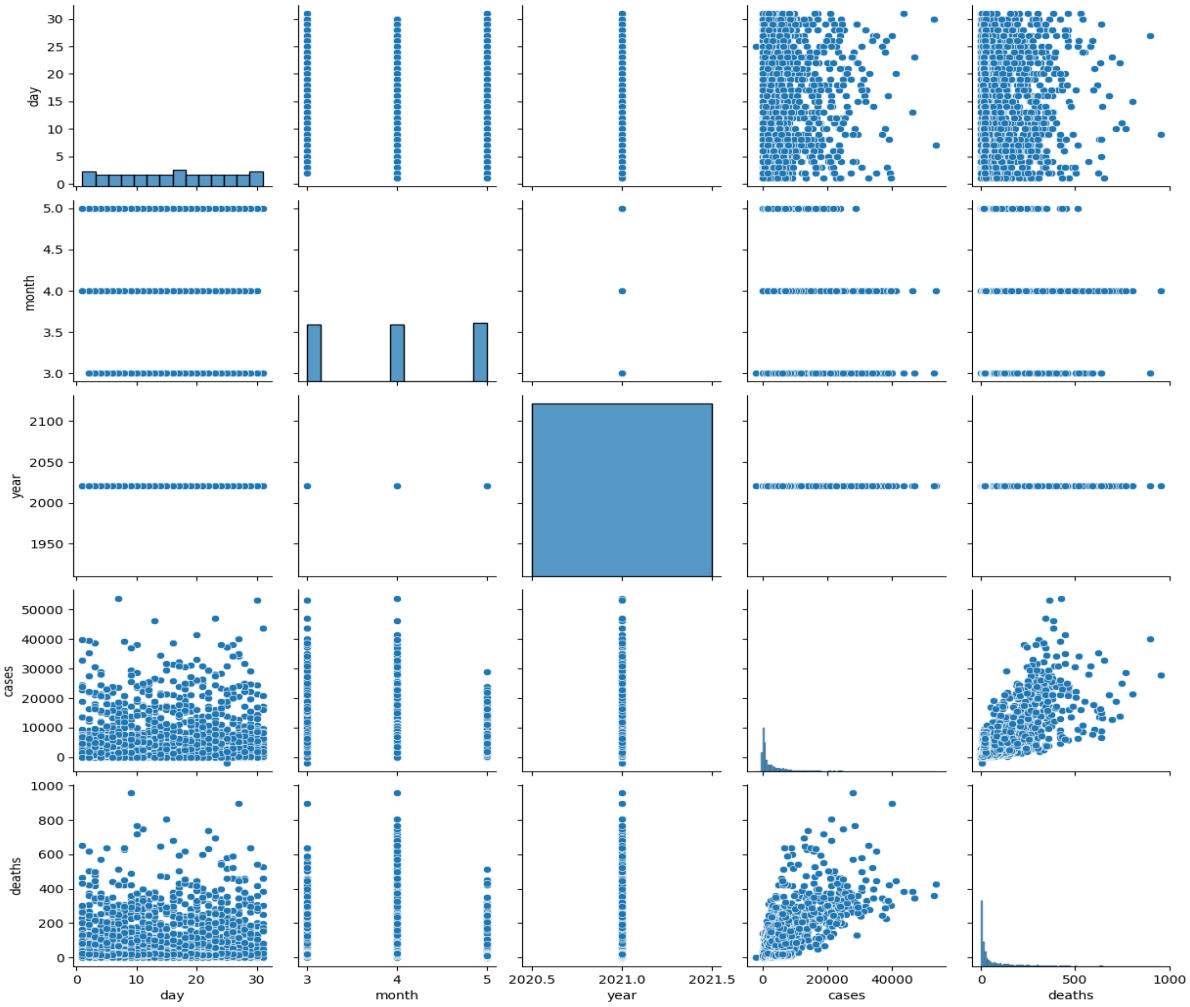
```
[0.3757015  0.3397918  0.34922823 0.3541712  0.33786297 0.3363355
 0.32468745 0.3000098  0.39277244 0.34725806 0.30215907 0.3213253
 0.32336944 0.3290632  0.3649855  0.35019916 0.39282632 0.33576846
 0.32892165 0.3184401  0.32449907 0.31502837 0.38507205 0.36293718
 0.30839074 0.36987197 0.30124998 0.3932455  0.3221074  0.35596192
 0.30732802 0.34227586 0.37582952 0.39523134 0.37292984 0.38207203
 0.32293206 0.3985834  0.37436926 0.34277743 0.31177938 0.36159268
 0.3558107  0.34616998 0.3187251  0.3340276  0.3442348  0.3454052
 0.39061412 0.37812942 0.34791535 0.3665589  0.32024652 0.32946324
 0.37966317 0.36126342 0.359875   0.34309894 0.33145776 0.31612667
 0.37237984 0.34349003 0.39279705 0.31256002 0.34508586 0.38956803
 0.3138677  0.38808697 0.39639476 0.34673712 0.32411188 0.39526606
 0.39624074 0.3824527  0.3263294  0.30289352 0.3315716  0.3168191
 0.3594942  0.33065638 0.39431536 0.35150912 0.32562375 0.34389246
 0.30213296 0.36899322 0.38280836 0.34653014 0.31952286 0.39911014
 0.37387967 0.37827012 0.3951025  0.3939395  0.33668992 0.32887208
 0.32550773 0.37824243 0.33653998 0.39663807]
```

Module 6:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

df = pd.read_csv('D:/Covid_19_cases.csv')
print(df.head())
    dateRep  day  month  year  cases  deaths countriesAndTerritories
0  31-05-2021   31     5  2021    366      5               Austria
1  30-05-2021   30     5  2021    570      6               Austria
2  29-05-2021   29     5  2021    538     11                Austria
3  28-05-2021   28     5  2021    639      4               Austria
4  27-05-2021   27     5  2021    405     19                Austria

X = df['day'].values.reshape(-1, 1)
y = df['cases'].values.reshape(-1, 1)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)

lr = LinearRegression()
lr.fit(X_train, y_train)

y_pred_train = lr.predict(X_train)
y_pred_test = lr.predict(X_test)

print('Train Mean Squared Error:', mean_squared_error(y_train, y_pred_train))
print('Train R-squared Score:', r2_score(y_train, y_pred_train))

print('Test Mean Squared Error:', mean_squared_error(y_test, y_pred_test))
print('Test R-squared Score:', r2_score(y_test, y_pred_test))
Train Mean Squared Error: 44298169.61356584
Train R-squared Score: 0.0002403407098547916
Test Mean Squared Error: 33286949.375746086
Test R-squared Score: -0.005195049936161045
```

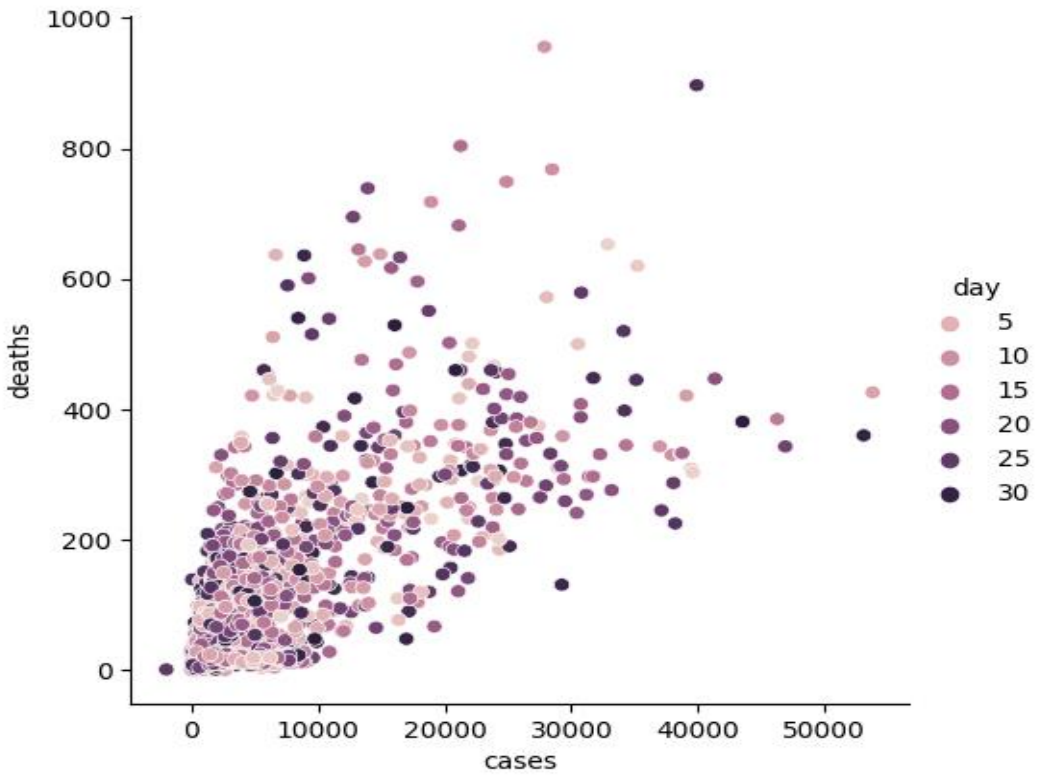
```
plt.scatter(X_test, y_test, color='black')
plt.plot(X_test, y_pred_test, color='blue', linewidth=3)
plt.xlabel('day')
plt.ylabel('COVID-19 Cases')
plt.title('COVID-19 Cases Prediction')
plt.show()
```





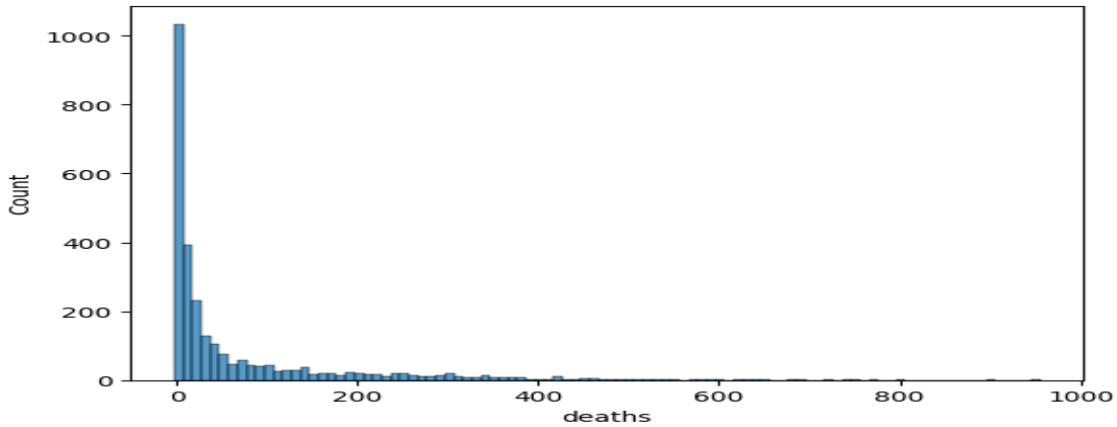
sns.relplot(x= 'cases', y='deaths', hue='day', data= Covid_19_cases)

<seaborn.axisgrid.FacetGrid at 0x1b46cd15110>



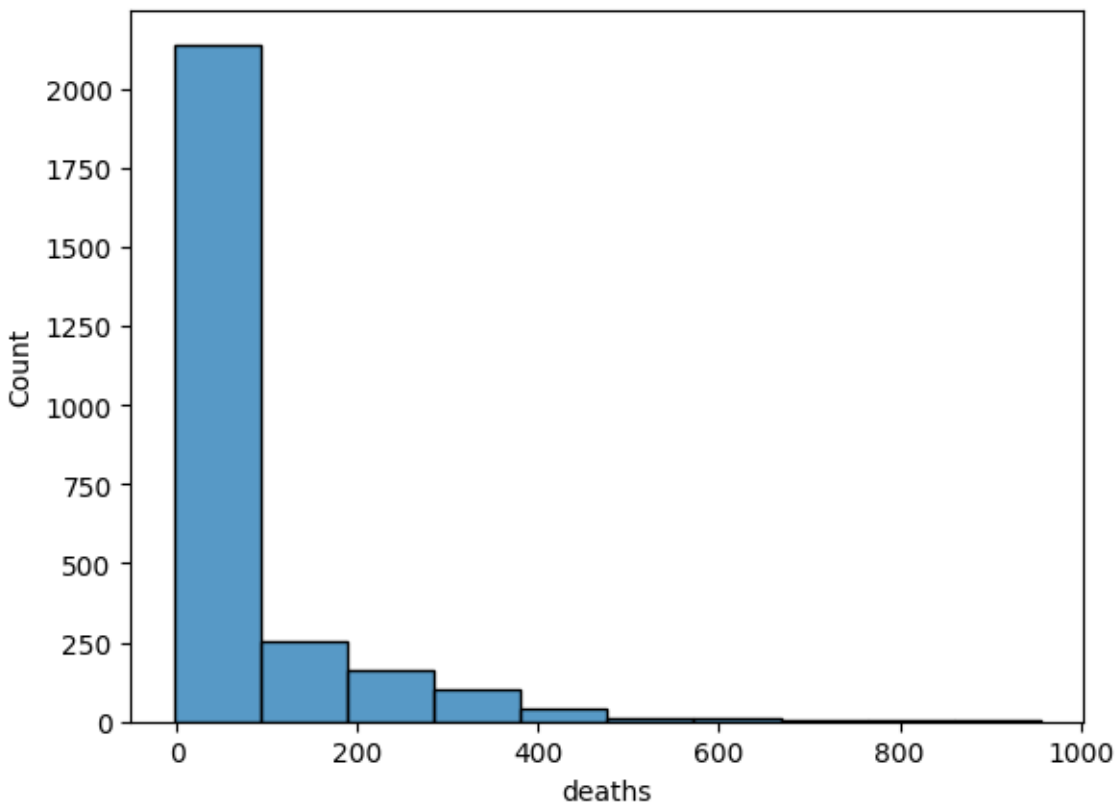
sns.histplot(Covid_19_cases['deaths'])

<Axes: xlabel='deaths', ylabel='Count'>

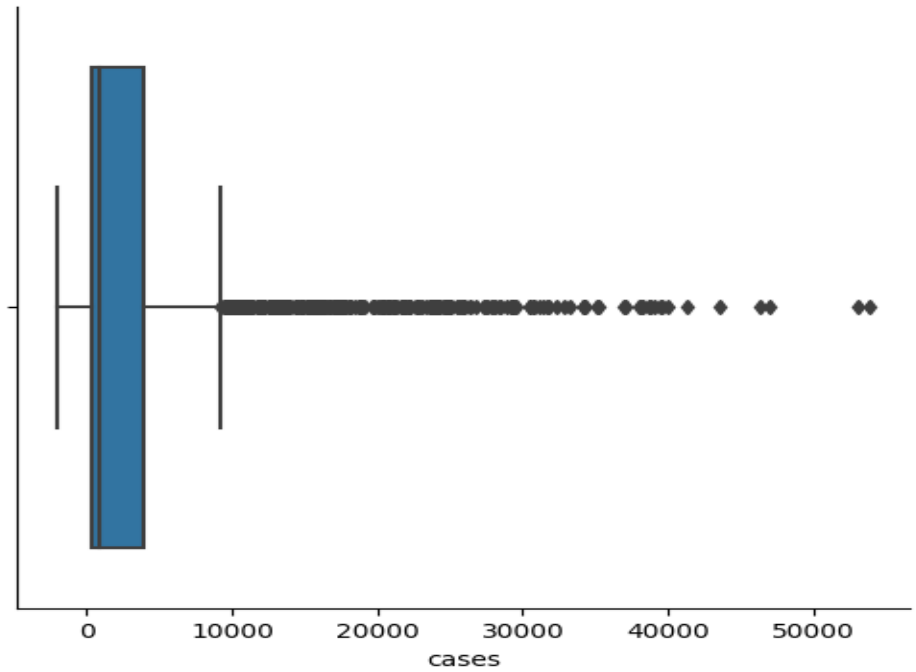


sns.histplot(Covid_19_cases['deaths'], bins=10)

<Axes: xlabel='deaths', ylabel='Count'>




```
sns.catplot(x='cases', kind= 'box', data =Covid_19_cases )
```

Out[31]:

```
<seaborn.axisgrid.FacetGrid at 0x1b4740dbf90>
```